



PROGRAMAÇÃO FRONT END
REACT

Prof. Esp. Dacio F. Machado



OBJETIVO DA AULA

→ manipulando elementos com **HTML** com **JS**
de um jeito um pouco diferente...

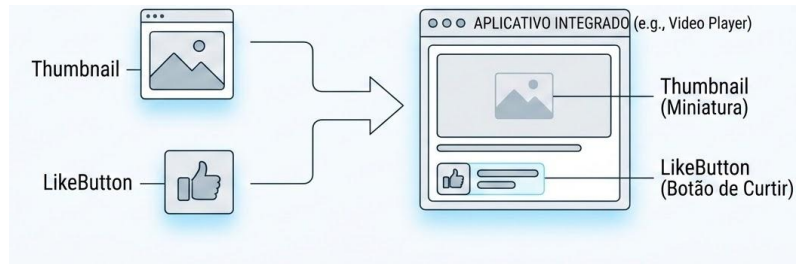


REACT

O React permite que você construa interfaces de usuário a partir de pedaços individuais chamados componentes.

Crie seus próprios componentes React, como Thumbnail (Miniatura), LikeButton (Botão de Curtir) e Video. Depois, combine-os em telas, páginas e aplicativos inteiros.

Ele foi projetado para permitir que você combine de forma fluida componentes escritos por pessoas, equipes e organizações independentes.





REACT

Criando e aninhando componentes

Os aplicativos React são feitos de componentes. Um componente é um pedaço da interface do usuário (UI) que possui sua própria lógica e aparência. Um componente pode ser tão pequeno quanto um botão ou tão grande quanto uma página inteira.

Os componentes React são funções JavaScript que retornam marcação (markup):



```
function MyButton() {  
  return (  
    <button>I'm a button</button>  
  );  
}  
  
export default function MyApp() {  
  return (  
    <div>  
      <h1>Welcome to my app</h1>  
      <MyButton />  
    </div>  
  );  
}
```



Renderização condicional

No React, não existe uma sintaxe especial para escrever condições. Em vez disso, você usará as mesmas técnicas que utiliza ao escrever código JavaScript comum. Por exemplo, você pode usar uma instrução `if` para incluir JSX condicionalmente

Renderizando listas

Você contará com recursos do JavaScript como o laço `for` e a função `map()` de arrays para renderizar listas de componentes.



Respondendo a eventos Você pode responder a eventos declarando funções manipuladoras de eventos (*event handlers*) dentro dos seus componentes

Atualizando a tela Frequentemente, você vai querer que seu componente "lembre" de alguma informação e a exiba. Por exemplo, talvez você queira contar o número de vezes que um botão é clicado. Para fazer isso, adicione um estado (*state*) ao seu componente. Você obterá duas coisas do **useState**: o estado atual (**count**) e a função que permite atualizá-lo (**setCount**). Você pode dar a eles qualquer nome, mas a convenção é escrever [**algo**, **setAlgo**].



APLICANDO



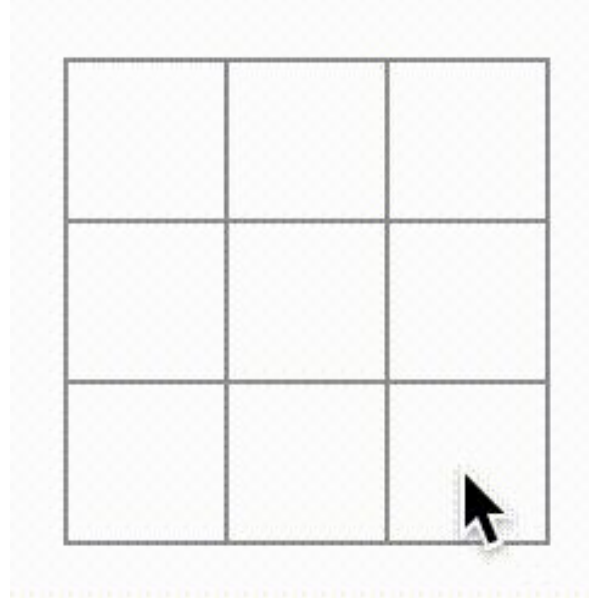


Aplicando React em um Projeto

Seguindo o exemplo da aula*

você aprenderá:

- **os fundamentos** do React:
componentes, props e state
- você aprenderá **as técnicas mais comuns** no desenvolvimento com React
- proporcionará **uma melhor e mais profunda compreensão**, do React

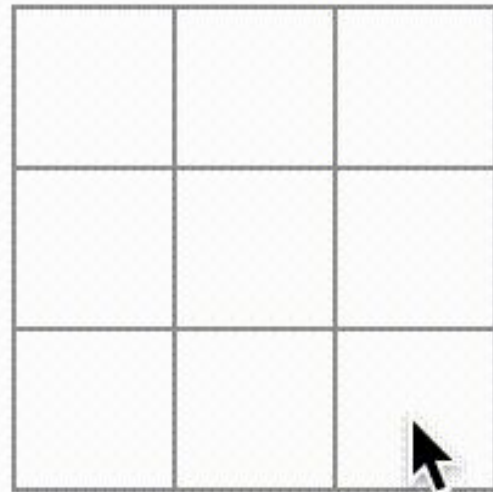




Aplicando React em um Projeto

Se o código ainda não fizer sentido para você, ou se você não estiver familiarizado com a sintaxe do código, não se preocupe! (isso não vai mudar tão cedo...)

O objetivo desta aula é ajudá-lo a entender o React e sua sintaxe.

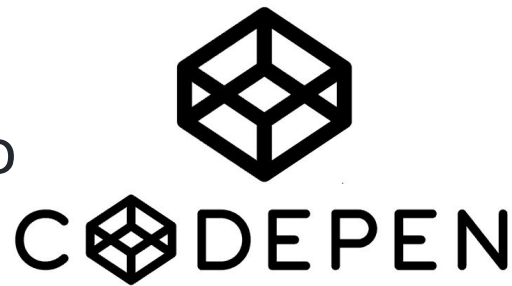


Vamos para o jogo da velha interativo com React



Pegando na sua caneta de código

CodePen permite que você escreva código no seu navegador e visualize como seus usuários verão o aplicativo que você criou



Contudo para executar o JSX, é necessário mudar, o preprocessor da ferramenta para o “babel”.

Pode fazer isso no menu de configurações da aba do JS do editor;

Agora que você já configurou tudo, vamos dar uma olhada geral no React!



```
import React from "https://esm.sh/react";
import ReactDOM from "https://esm.sh/react-dom";
import { StrictMode } from "https://esm.sh/react";
import { createRoot } from "https://esm.sh/react-dom/client";
import { useState } from;

const root = createRoot(document.getElementById("root"));
root.render(
  <StrictMode>
    <Game />
  </StrictMode>
);
```



Farmando o básico de Áurea

As linhas primeiras linhas de código reúnem todas as peças necessárias:

- Reagir (Biblioteca React)
- Biblioteca do React para comunicação com navegadores web (React DOM)

O restante do arquivo, serve de ponte entre o componente que você criou no navegador da web, pois reúne todas os componentes e insere o produto final `index.html`

Agora vamos dar uma olhada nos arquivos do código inicial.



```
import React from "https://esm.sh/react";
import ReactDOM from "https://esm.sh/react-dom";
import { StrictMode } from "https://esm.sh/react";
import { createRoot } from "https://esm.sh/react-dom/client";
import { useState } from;
```

```
function Square() {
  return <button className="square">X</button>;
}
```

```
const root = createRoot(document.getElementById("root"));
root.render(
```



Componentizando

No React, um componente é um trecho de código reutilizável que representa uma parte da interface do usuário. Os componentes são usados para renderizar, gerenciar e atualizar os elementos da interface do usuário em sua aplicação. Vamos analisar o componente linha por linha para ver o que está acontecendo:

```
function Square() {  
  return <button className="square">X</button>;  
}
```

A primeira linha define uma função chamada `Square`



Componentizando

```
function Square() {  
  return <button className="square">X</button>;  
}
```

A segunda linha retorna um botão. A palavra-chave `return` do JavaScript significa que o que quer que venha a seguir é retornado como um valor para quem chamou a função. `<button>` é um elemento JSX. Um elemento JSX é uma combinação de código JavaScript e tags HTML que descreve o que você gostaria de exibir. `className="square"` é uma propriedade do botão, ou *prop*, que diz ao CSS como estilizar o botão. `X` é o texto exibido dentro do botão e `</button>` fecha o elemento JSX para indicar que nenhum conteúdo seguinte deve ser colocado dentro do botão.



Componentizando

```
function Square() {  
  return <button className="square">X</button>;  
}
```

Ao inserir essa função dentro do nosso código e chamá-la como o componente a ser renderizado pelo root A seção do navegador deve exibir um quadrado com um X dentro, assim:





Construindo o tabuleiro

Vamos voltar ao código;

Atualmente, o tabuleiro tem apenas um quadrado, mas você precisa de nove!

Se você tentar copiar e colar o seu quadrado para criar dois quadrados como retorno da função, um após o outro vai tomar na cara o erro:

```
/src/App.js: Adjacent JSX elements must be wrapped in  
an enclosing tag. Did you want a JSX Fragment  
<>...</>?
```



Construindo o tabuleiro

Os componentes React precisam retornar um único elemento JSX e não vários elementos JSX adjacentes, como dois botões.

Para corrigir isso, você pode usar *Fragments* (`<>` e `</>` e `<String>`) para envolver vários elementos JSX adjacentes, vamos atualizar a função `Square()`

```
function Square() {  
  return (  
    <>  
      <button className="square">X</button>  
      <button className="square">X</button>  
    </>  
  );  
}
```




Construindo o tabuleiro

Oh não!

Os quadrados estão todos em uma única linha, e não em uma grade como você precisa para o nosso tabuleiro.

Para corrigir isso, você precisará agrupar seus quadrados em linhas com uma `div` e adicionar algumas classes CSS.

Já que está fazendo isso, você também pode atribuir um número a cada quadrado para saber exatamente onde cada um será exibido.

No `App.js` arquivo, atualize o `Square` componente para que fique assim:



```
function Square() {  
  return (  
    <>  
      <div className="board-row">  
        <button className="square">1</button>  
        <button className="square">2</button>  
        <button className="square">3</button>  
      </div>  
      <div className="board-row">  
        <button className="square">4</button>  
        <button className="square">5</button>  
        <button className="square">6</button>  
      </div>  
      <div className="board-row">  
        <button className="square">7</button>  
        <button className="square">8</button>  
        <button className="square">9</button>  
      </div>  
    </>  
  );  
}
```

```
.square {  
  background: #fff;  
  border: 1px solid #999;  
  float: left;  
  font-size: 24px;  
  font-weight: bold;  
  line-height: 34px;  
  height: 34px;  
  margin-right: -1px;  
  margin-top: -1px;  
  padding: 0;  
  text-align: center;  
  width: 34px;  
}
```



Construindo o tabuleiro

O CSS definido em `styles.css`styles`` estiliza as `div`s com o `className` atributo ``style` board-row`. Agora que você agrupou seus componentes em linhas com os `div` elementos estilizados, você tem seu tabuleiro de jogo da velha:

1	2	3
4	5	6
7	8	9

Mas agora você tem um problema. Seu componente chamado `Square`, na verdade não é mais um quadrado. Vamos corrigir isso mudando o nome para `Board`: `function Board() { ... }`



Só pra constar

Observação

Psiu... É muita coisa para digitar! Você pode copiar e colar o código desta página sem problemas. No entanto, se quiser um pequeno desafio, recomendamos copiar apenas códigos que você já tenha digitado manualmente pelo menos uma vez.



Passando dados através de props

Em seguida, você vai querer alterar o valor de um quadrado de vazio para "X" quando o usuário clicar nele.

Com a forma como você construiu o tabuleiro até agora, você precisaria copiar e colar o código que atualiza o quadrado nove vezes (uma vez para cada quadrado)!

Em vez de copiar e colar, a arquitetura de componentes do React permite que você crie um componente reutilizável para evitar código duplicado e confuso.



Passando dados através de props

Para isso vamos basicamente reconstruir o componente `Square()` e utilizá-lo para construir o nosso tabuleiro `Board()`.

```
function Square() {  
  return <button className="square">X</button>;  
}
```

Onde nosso tabuleiro será um componente composto construído com:

```
function Board() {  
  return (  
    <>  
      <div className="board-row">  
        <Square />  
        <Square />  
        <Square />  
      </div>  
    )  
  )  
}
```



Passando dados através de props

Observe que, diferentemente do divisor `div`, seus próprios componentes `Board` **devem** `Square` começar com letra maiúscula.

Vamos dar uma olhada, como ficou nosso tabuleiro:

1	1	1
1	1	1
1	1	1



Passando dados através de props

Oh não! Você perdeu os quadrados numerados que tinha antes. Agora cada quadrado diz “1”. Para corrigir isso, você usará **props** para passar o valor que cada quadrado deve ter do componente pai (`Board`) para o seu componente filho (`Square`).

[Abre aspas do prof : **props** é um nome bonitinho para um parâmetro]

Atualize o `Square` componente para ler a `value` propriedade que você passará pelo `Board`:

```
function Square({ value }) {  
  return <button className="square">1</button>;  
}
```

`function Square({ value })` Indica que o componente `Square` pode receber uma propriedade chamada `value`.



Passando dados através de props

Agora você quer exibir o nosso parâmetro `value` em vez de `1` dentro de cada quadrado.

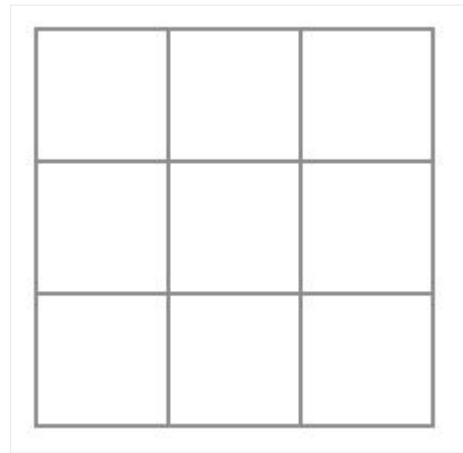
Lembre-se que se você queria renderizar a variável JavaScript chamada `value` pelo seu componente, não a palavra "value". Para "escapar para JavaScript" a partir do JSX, você precisa de chaves. Adicione chaves ao redor do código `{value}`

```
function Square({ value }) {  
  return <button className="square">{value}</button>;  
}
```



Passando dados através de props

Por enquanto, você deverá ver um tabuleiro vazio:



Isso ocorre porque o `Board` componente ainda não passou a `value` propriedade para cada `Square` componente que renderiza.

Para corrigir isso, você deve adicionar a `value` propriedade a cada `Square` componente renderizado pelo `Board` componente:

```
<div className="board-row">  
  <Square value="1" />  
  <Square value="2" />  
  <Square value="3" />  
</div>
```



Passando dados através de props

Agora você deverá ver novamente uma grade de números:

1	2	3
4	5	6
7	8	9

Seu código atualizado deve ficar assim:



```
function Square({ value }) {  
  return <button className="square">{value}</button>;  
}  
  
export default function Board() {  
  return (  
    <>  
      <div className="board-row">  
        <Square value="1" />  
        <Square value="2" />  
        <Square value="3" />  
      </div>  
      <div className="board-row">  
        <Square value="4" />  
        <Square value="5" />  
        <Square value="6" />  
      </div>  
      <div className="board-row">  
        <Square value="7" />  
        <Square value="8" />  
        <Square value="9" />  
      </div>  
    </>  
  );  
}
```



Criando um componente interativo





```
function Square({ value, onSquareClick }) {  
  return (  
    <button className="square" onClick={onSquareClick}>  
      {value}  
    </button>  
  );  
}
```

```
function Board({ xIsNext, squares, onPlay }) {  
  function handleClick(i) {  
    if (calculateWinner(squares) || squares[i]) {  
      return;  
    }  
    const nextSquares = squares.slice();  
    if (xIsNext) {  
      nextSquares[i] = 'X';  
    } else {  
      nextSquares[i] = 'O';  
    }  
    onPlay(nextSquares);  
  }  
}
```

```
const winner = calculateWinner(squares);  
let status;  
if (winner) {  
  status = 'Winner: ' + winner;  
} else {
```



OBRIGADO

daciofmf@gmail.com